

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.dal.function-`
Release 1.0 SNAPSHOT

September 2026



Copyright © 2000, 2026 Eclipse Foundation

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
142	Device Abstraction Layer Functions Specification	7
142.1	Introduction	7
142.2	Functions	7
142.3	Functions Data	12
142.4	org.osgi.service.dal.functions	14
142.5	org.osgi.service.dal.functions.data	26

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

142 Device Abstraction Layer Functions Specification

Version 1.0

142.1 Introduction

Concrete function interfaces are used to unify the access and the control of the basic device operations and the related properties. The current section specifies the minimal set of such functionalities. They can be extended or replaced to cover domain specific scenarios. The set is not closed and can be incorporated with vendor specific functions. There is support for: control, monitoring and metering information.

142.2 Functions

142.2.1 BooleanControl

BooleanControl function provides a binary control support. The property eventing must follow the definition of Device Abstraction Layer, ???. The full function definition is available in the next tables.

Table 142.1 BooleanControl Operations

Name	Description
inverse	Reverses the BooleanControl state. If the current state represents true value, it'll be changed to false. If the current state represents false value, it'll be changed to true.
setTrue	Sets the BooleanControl state to true value.
setFalse	Sets the BooleanControl state to false value.

Table 142.2 BooleanControl Properties

Name	Description
data	Contains the current state of BooleanControl. The property access is readable, writable and eventable.

Different types can be used as a value of ??? service property. The next list contains some suitable to BooleanControl:

- **LIGHT** - indicates that there is a light device control. true state means that the light device will be turned on. false state means that the light device will be turned off.
- **DOOR** - indicates that there is a door position control. true state means that the door will be opened. false state means that the door will be closed.

- **WINDOW** - indicates that there is a window position control. true state means that the window will be opened. false state means that the window will be closed.
- **POWER** - indicates that there is electricity control. true state means that the power will be re-stored. false state means that the power will be cut.
- other type defined in **Types**
- vendor specific

The function is using *BooleanData* on page 12 data structure to provide the control state.

The next code snippet sets to true all BooleanControl functions, which control the light.

```
ServiceReference[] booleanControlsRefs = context.getServiceReferences(
    BooleanControl.class.getName(),
    '(' + Function.SERVICE_TYPE + '=' + Types.LIGHT + ')');
if (booleanControlsRefs != null) {
    for (int i = 0; i < booleanControlsRefs.length; i++) {
        BooleanControl booleanControl = (BooleanControl) context.getService(
            booleanControlsRefs[i]);
        if (booleanControl != null) {
            booleanControl.setTrue();
            context.ungetService(booleanControlsRefs[i]);
        }
    }
}
```

142.2.2

BooleanSensor

BooleanSensor function provides binary sensor monitoring. It reports the state when an important event is available. There are no operations. The property eventing must follow the definition of Device Abstraction Layer, ??? The full function definition is available in the next table.

Table 142.3 BooleanSensor Properties

Name	Description
data	Contains the current state of BooleanSensor. The property access is readable and eventable.

Different types can be used as a value of ??? service property. The next list contains some suitable to BooleanSensor:

- **LIGHT** - indicates that the BooleanSensor can detected light. true state means that there is light. false state means that there is no light.
- **GAS** - indicates that the BooleanSensor supports gas detection. true state means there is gas. false state means that there is no gas.
- **SMOKE** - indicates that the BooleanSensor can detect smoke. true state means that there is smoke. false state means that there is no smoke.
- **DOOR** - indicates that the BooleanSensor can detect the door state. true state means that the door is opened. false state means that the door is closed.
- **WINDOW** - indicates that the BooleanSensor can window state. true state means that the window is opened. false state means that the window is closed.
- **POWER** - indicates that the BooleanSensor can detect power/no power. true state means that there is power. false state means that there is no power.
- **RAIN** - indicates that the BooleanSensor can detect rain. true state means that there is rain. false state means that there is no rain.

- **CONTACT** - indicates that the BooleanSensor can detect contact. true state means that there is contact. false state means that there is no contact.
- **FIRE** - indicates that the BooleanSensor can detect fire. true state means that there is fire. false state means that there is no fire.
- **OCCUPANCY** - indicates that the BooleanSensor can detect presence. true state means that someone is detected. false state means that nobody is detected.
- **WATER** - indicates that the BooleanSensor can detect water leak. true state means that there is water leak. false state means that there is no water leak.
- **MOTION** - indicates that the BooleanSensor can detect motion. true state means that there is motion detection. false state means that there is no motion detection.
- other type defined in [Types](#)
- vendor specific

The function is using *BooleanData* on page 12 data structure to provide the sensor state.

142.2.3 MultiLevelControl

MultiLevelControl function provides multi-level control support. The property eventing must follow the definition of Device Abstraction Layer, ????. The full function definition is available in the next table.

Table 142.4 *MultiLevelControl Properties*

Name	Description
data	Contains the current state of MultiLevelControl. The property access is readable, writable and eventable.

Different types can be used as a value of ??? service property. The next list contains some suitable to MultiLevelControl:

- **LIGHT** - indicates that the MultiLevelControl can control light devices. Usually, such devices are called dimmable. MultiLevelControl minimum value can switch off the device and MultiLevelControl maximum value can increase the device light to the maximum possible value.
- **TEMPERATURE** - indicates that the MultiLevelControl can control temperature devices. For example, such device can be thermostat. MultiLevelControl minimum value is the lowest supported temperature. MultiLevelControl maximum value is the highest supported temperature.
- **FLOW** - indicates that the MultiLevelControl can control the flow level. MultiLevelControl minimum value is the minimum supported flow level. MultiLevelControl maximum value is the maximum supported flow level.
- **PRESSURE** - indicates that the MultiLevelControl can control the pressure level. MultiLevelControl minimum value is the lowest supported pressure level. MultiLevelControl maximum value is the highest supported pressure level.
- **HUMIDITY** - indicates that the MultiLevelControl can control the humidity level. It's typical functionality for HVAC (heating, ventilation, and air conditioning) devices. MultiLevelControl minimum value is the lowest supported humidity level. MultiLevelControl maximum value is the highest supported humidity level.
- **GAS** - indicates that the MultiLevelControl can control the gas level. MultiLevelControl minimum value is the lowest supported gas level. MultiLevelControl maximum value is the highest supported gas level.
- **SMOKE** - indicates that the MultiLevelControl can control the smoke level. MultiLevelControl minimum value is the lowest supported smoke level. MultiLevelControl maximum value is the highest supported smoke level.

- **DOOR** - indicates that the MultiLevelControl can control the door position. MultiLevelControl minimum value can completely close the door. MultiLevelControl maximum value can open the door to the maximum allowed position.
- **WINDOW** - indicates that the MultiLevelControl can control the window position. MultiLevelControl minimum value can completely close the window. MultiLevelControl maximum value can open the window to the maximum allowed position.
- **LIQUID** - indicates that the MultiLevelControl can control the liquid level. MultiLevelControl minimum value is the lowest supported liquid level. MultiLevelControl maximum value is the highest supported liquid level.
- **POWER** - indicates that the MultiLevelControl can control the power level. MultiLevelControl minimum value is the lowest supported power level. MultiLevelControl maximum value is the highest supported power level.
- **NOISINESS** - indicates that the MultiLevelControl can control the noise level. MultiLevelControl minimum value is the lowest supported noise level. MultiLevelControl maximum value is the highest supported noise level.
- other type defined in [Types](#)
- vendor specific

The function is using *LevelData* on page 13 data structure to provide the level.

142.2.4

MultiLevelSensor

MultiLevelSensor function provides multi-level sensor monitoring. It reports its state when an important event is available. There are no operations. The property eventing must follow the definition of Device Abstraction Later, ???. The full function definition is available in the next table.

Table 142.5

MultiLevelSensor Properties

Name	Description
data	Contains the current state of MultiLevelSensor. The property access is readable and eventable.

Different types can be used as a value of ??? service property. The next list contains some suitable to MultiLevelSensor:

- **LIGHT** - indicates that the sensor can monitor the light level.
- **TEMPERATURE** - indicates that the sensor can monitor the temperature.
- **FLOW** - indicates that the sensor can monitor the flow level.
- **PRESSURE** - indicates that the sensor can monitor the pressure level.
- **HUMIDITY** - indicates that the sensor can monitor the humidity level.
- **GAS** - indicates that the sensor can monitor the gas level.
- **SMOKE** - indicates that the sensor can monitor the smoke level.
- **DOOR** - indicates that the sensor can monitor the door position.
- **WINDOW** - indicates that the sensor can monitor the window position.
- **LIQUID** - indicates that the sensor can monitor the liquid level.
- **POWER** - indicates that the sensor can monitor the power level.
- **NOISINESS** - indicates that the sensor can monitor the noise level.
- **RAIN** - indicates that the MultiLevelSensor can monitor the rain rate.
- other type defined in [Types](#)
- vendor specific

The function is using *LevelData* on page 13 data structure to provide the level.

142.2.5 Meter

[Meter](#) function can measure metering information. It provides the current and total consumptions or generations. The property eventing must follow the definition of Device Abstraction Layer, ???. The full function definition is available in the next tables.

Table 142.6 *Meter Properties*

Name	Description
total	Contains the total consumption or production. The property access is readable and eventable.
current	Contains the current consumption or production. The property access is readable and eventable.

Different types can be used as a value of [??? service property](#). The next list contains some suitable to Meter:

- [PRESSURE](#) - indicates that the Meter measures pressure.
- [GAS](#) - indicates that the Meter measures the gas consumption.
- [POWER](#) - indicates that the Meter measures the power consumption.
- [WATER](#) - indicates that the Meter measures water consumption.
- [HEAT](#) - indicates that the Meter measures thermal energy provided by a source.
- [COLD](#) - indicates that the Meter measures thermal energy provided by a source.
- other type defined in [Types](#)
- vendor specific

The function is using [LevelData](#) on page 13 data structure to provide metering information.

Meter function service can be optionally registered with [SERVICE_FLOW](#) service property. The value type is java.lang.String. It contains the metering flow. Currently, the flow can be [FLOW_IN](#) for a consumption or [FLOW_OUT](#) for a production.

142.2.6 Alarm

[Alarm](#) function provides alarm sensor support. There is only one eventable property and no operations. The property eventing must follow the definition of Device Abstraction Layer, ???. The full function definition is available in the next table.

Table 142.7 *BooleanSensor Properties*

Name	Description
alarm	Specifies the alarm property name. The property is eventable.

The function is using [AlarmData](#) on page 13 data structure to report the alarm. The property eventing must follow the definition of Device Abstraction Layer, ???.

142.2.7 Keypad

[Keypad](#) function provides support for keypad control. The keypad typically consists of one or more keys/buttons, which can be discerned. Different types of key presses like short and long press can typically also be detected. Each key pressed event is followed by a key released event. It's not possible to have two consecutive key pressed or key released events. There is only one eventable property and no operations. The property eventing must follow the definition of Device Abstraction Layer, ???. The full function definition is available in the next table.

Table 142.8 Keypad Properties

Name	Description
key	Specifies a property name for a key from the keypad. The property is eventable.
The function is using <i>KeypadData</i> on page 13 data structure to report the keys.	

142.2.8 WakeUp

[WakeUp](#) function provides device awake monitoring. It's especially applicable to battery-operated devices. Such device can notify the system that it's awake and can receive commands with a [PROPERTY_AWAKE](#) property event. The property eventing must follow the definition of Device Abstraction Layer, ???.

The device can periodically wake up for commands. The interval can be managed with [PROPERTY_WAKE_UP_INTERVAL](#) property.

Table 142.9 WakeUp Properties

Name	Description
awake	Specifies the awake eventable property name. If the device is awake, it will trigger a property event. The property value type is <i>BooleanData</i> on page 12 .
wakeUpInterval	Specifies the wake up interval. The device can periodically wake up and receive commands. That interval is managed by this property. The property access is readable, writable and eventable. The property value type is <i>LevelData</i> on page 13 .

142.3 Functions Data

??? subclasses are wrappers on top of the java types to cover the requirements of the Device Abstraction Layer section. They can be received with the getter methods, can be set with the setter methods and can be reported with ???. The value can be described with different properties like:

- timestamp - the timestamp is the difference between the value collecting time and midnight, January 1, 1970 UTC. It's measured in milliseconds. The device driver is responsible to generate that value when the value is received from the device.
- unit - represents the value unit as it's defined in ???.
- description - represents a human readable description of the value.

142.3.1 BooleanData

[BooleanData](#) is used by *BooleanControl* on page 7 , *BooleanSensor* on page 8 and *WakeUp* on page 12 .

It provides information about the function state. That data object contains boolean value, the value collecting time and additional metadata. The value field is accessible with [getValue\(\)](#) getter. Other fields are inherited from the parent class ???.

Two *BooleanData* instances are equal if they contain equal metadata, timestamp and boolean value. [compareTo\(Object\)](#) method compares *BooleanData* instance with the given argument of the same type and returns:

- -1 if the instance field is less than a field of the specified argument.
- 0 if all fields are equivalent.
- 1 if the instance field is greater than a field of the specified argument.

The fields are compared in this order: timestamp, metadata, value.

142.3.2 LevelData

[LevelData](#) is used by *MultiLevelControl* on page 9 , *MultiLevelSensor* on page 10 , *Meter* on page 11 and *WakeUp* on page 12 .

It provides information about the function level. That data object contains BigDecimal value and the value unit. The measurement unit is used as it's defined in ??? . The unit field is accessible with [getUnit\(\)](#) getter. The level field is accessible with [getLevel\(\)](#) getter.

Two LevelData instances are equal if they contain equal metadata, timestamp, unit and level.

[compareTo\(Object\)](#) method compares LevelData instance with the given argument of the same type and returns:

- -1 if the instance field is less than a field of the specified argument.
- 0 if all fields are equivalent.
- 1 if the instance field is greater than a field of the specified argument.

The fields are compared in this order: timestamp, metadata, level, unit.

142.3.3 AlarmData

[AlarmData](#) is used by *Alarm* on page 11 .

AlarmData data structure is used to provide information about the available alarm. That data object contains:

- alarm type - indicates the meaning of the alarm like smoke, power fail, etc.
- alarm severity - indicates the alarm importance level like minor, critical, etc.

The severity field is accessible with [getSeverity\(\)](#) getter. The type field is accessible with [getType\(\)](#) getter.

Two AlarmData instances are equal if they contain equal metadata, timestamp, type and severity.

[compareTo\(Object\)](#) method compares AlarmData instance with the given argument of the same type and returns:

- -1 if the instance field is less than a field of the specified argument.
- 0 if all fields are equivalent.
- 1 if the instance field is greater than a field of the specified argument.

The fields are compared in this order: timestamp, metadata, type, severity.

142.3.4 KeypadData

[KeypadData](#) is used by *Keypad* on page 11 .

KeypadData data structure is used to provide information when a change with some key from the keypad has occurred. That data object contains the event type, sub-type, key code and key name. Currently, there are two predefined event types:

- [TYPE_PRESSED](#) – used for a key pressed;
- [TYPE_RELEASED](#) – used for a key released.

Predefined event sub-types are:

- `SUB_TYPE_PRESSED_NORMAL` – used for a normal key pressed event. Usually, there is a single press and the key is not held down. This sub-type is used with `TYPE_PRESSED` type.
- `SUB_TYPE_PRESSED_LONG` – used for a long key pressed event. Usually, there is a single press and the key is held down. This sub-type is used with `TYPE_PRESSED` type.
- `SUB_TYPE_PRESSED_DOUBLE` – used for a double key pressed event. Usually, there are two press actions and the key is not held down after the second press. This sub-type is used with `TYPE_PRESSED` type.
- `SUB_TYPE_PRESSED_DOUBLE_LONG` – used for a double long key pressed event. Usually, there are two press actions and the key is held down after the second press. This sub-type is used with `TYPE_PRESSED` type.

The type field is accessible with `getType()` getter. The subType field is accessible with `getSubType()` getter. The keyCode field is accessible with `getKeyCode()` getter. The keyName field is accessible with `getKeyName()` getter.

Two KeypadData instances are equal if they contain equal metadata, timestamp, event type, sub-type, key code and key name.

`compareTo(Object)` method compares KeypadData instance with the given argument of the same type and returns:

- -1 if the instance field is less than a field of the specified argument.
- 0 if all fields are equivalent.
- 1 if the instance field is greater than a field of the specified argument.

The fields are compared in this order: timestamp, metadata, type, sub-type, key code, key name.

142.4 org.osgi.service.dal.functions

Device Abstraction Layer Functions Package 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.dal.functions; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.dal.functions; version="[1.0,1.1)"}
```

142.4.1 Summary

- Alarm - {@code Alarm} function provides alarm sensor support.
- BooleanControl - {@code BooleanControl} function provides a boolean control support.
- BooleanSensor - {@code BooleanSensor} function provides boolean sensor monitoring.
- Keypad - {@code Keypad} function provides support for keypad control.
- Meter - {@code Meter} function can measure metering information.
- MultiLevelControl - {@code MultiLevelControl} function provides multi-level control support.
- MultiLevelSensor - {@code MultiLevelSensor} function provides multi-level sensor monitoring.
- Types - Shares common constants for all functions defined in this package.
- WakeUp - {@code WakeUp} function provides device awake monitoring.

142.4.2 **public abstract Alarm** **extends Function**

{@code Alarm} function provides alarm sensor support. There is only one eventable property and no operations.

See Also AlarmData

142.4.2.1 **public static final String PROPERTY_ALARM = "alarm"**

Specifies the alarm property name. The property is eventable.

See Also AlarmData

142.4.3 **public abstract BooleanControl** **extends Function**

{@code BooleanControl} function provides a boolean control support. The eventable function state is accessible with {@link #getData()} getter and {@link #setData(boolean)} setter. The state can be reversed with {@link #inverse()} method, can be set to {@code true} value with {@link #setTrue()} method and can be set to {@code false} value with {@link #setFalse()} method.

The control type can be:

- {@link Types#LIGHT}
- {@link Types#DOOR}
- {@link Types#WINDOW}
- {@link Types#POWER}
- other type defined in {@link Types}
- custom - vendor specific type

See Also BooleanData

142.4.3.1 **public static final String OPERATION_INVERSE = "inverse"**

Specifies the inverse operation name. The operation can be executed with {@link #inverse()} method.

142.4.3.2 **public static final String OPERATION_SET_FALSE = "setFalse"**

Specifies the operation name, which sets the control state to {@code false} value. The operation can be executed with {@link #setFalse()} method.

142.4.3.3 **public static final String OPERATION_SET_TRUE = "setTrue"**

Specifies the operation name, which sets the control state to {@code true} value. The operation can be executed with {@link #setTrue()} method.

142.4.3.4 **public static final String PROPERTY_DATA = "data"**

Specifies the state property name. The eventable property value is accessible with {@link #getData()} method.

See Also BooleanData

142.4.3.5 **public abstract BooleanData getData() throws DeviceException**

- Returns the current state of {@code BooleanControl}. It's a getter method for {@link #PROPERTY_DATA} property.

Returns The current state of {@code BooleanControl}.

Throws IllegalStateException – If this function service object has already been unregistered.

DeviceException— If an operation error is available.

See Also BooleanData, BooleanControl.PROPERTY_DATA

142.4.3.6 **public abstract void inverse() throws DeviceException**

- Reverses the {@code BooleanControl} state. If the current state represents {@code true} value, it'll be changed to {@code false}. If the current state represents {@code false} value, it'll be changed to {@code true}. The operation name is {@link #OPERATION_INVERSE}.

Throws IllegalStateException— If this function service object has already been unregistered.

DeviceException— If an operation error is available.

142.4.3.7 **public abstract void setData(boolean data) throws DeviceException**

data The new function value.

- Sets the {@code BooleanControl} state to the specified value. It's setter method for {@link #PROPERTY_DATA} property.

Throws IllegalStateException— If this function service object has already been unregistered.

DeviceException— If an operation error is available.

IllegalArgumentException— If there is an invalid argument.

See Also BooleanControl.PROPERTY_DATA

142.4.3.8 **public abstract void setFalse() throws DeviceException**

- Sets the {@code BooleanControl} state to {@code false} value. The operation name is {@link #OPERATION_SET_FALSE}.

Throws IllegalStateException— If this function service object has already been unregistered.

DeviceException— If an operation error is available.

142.4.3.9 **public abstract void setTrue() throws DeviceException**

- Sets the {@code BooleanControl} state to {@code true} value. The operation name is {@link #OPERATION_SET_TRUE}.

Throws IllegalStateException— If this function service object has already been unregistered.

DeviceException— If an operation error is available.

142.4.4 **public abstract BooleanSensor extends Function**

{@code BooleanSensor} function provides boolean sensor monitoring. It reports its state when an important event is available. The eventable state is accessible with {@link #getData()} getter. There are no operations.

The sensor type can be:

- {@link Types#LIGHT}
- {@link Types#GAS}
- {@link Types#SMOKE}
- {@link Types#DOOR}
- {@link Types#WINDOW}
- {@link Types#POWER}
- {@link Types#RAIN}
- {@link Types#CONTACT}

- {@link Types#FIRE}
- {@link Types#OCCUPANCY}
- {@link Types#WATER}
- {@link Types#MOTION}
- other type defined in {@link Types}
- custom - vendor specific type

See Also BooleanData

142.4.4.1 **public static final String PROPERTY_DATA = "data"**

Specifies the state property name. The eventable property value is accessible with {@link #getData()} getter.

142.4.4.2 **public abstract BooleanData getData() throws DeviceException**

- Returns the {@code BooleanSensor} current state. It's a getter method for {@link #PROPERTY_DATA} property.

Returns The {@code BooleanSensor} current state.

Throws IllegalStateException – If this function service object has already been unregistered.

DeviceException – If an operation error is available.

See Also BooleanData

142.4.5 **public abstract Keypad extends Function**

{@code Keypad} function provides support for keypad control. The keypad typically consists of one or more keys/buttons, which can be discerned. Different types of key presses like short and long press can typically also be detected. Each key pressed event is followed by a key released event. It's not possible to have two consecutive key pressed or key released events. There is only one eventable property and no operations.

{@code Keypad} can enumerate all supported keys in the key property metadata, {@link PropertyMetadata#getEnumValues(String)}.

See Also KeypadData

142.4.5.1 **public static final String PROPERTY_KEY = "key"**

Specifies a property name for a key from the keypad. The property is eventable.

See Also KeypadData

142.4.6 **public abstract Meter extends Function**

{@code Meter} function can measure metering information. The function provides these properties:

- {@link #PROPERTY_CURRENT} - eventable property accessible with {@link #getCurrent()} getter;
- {@link #PROPERTY_TOTAL} - eventable property accessible with {@link #getTotal()} getter.

The sensor type can be:

- {@link Types#PRESSURE}
- {@link Types#GAS}
- {@link Types#POWER}

- {@link Types#WATER}
- {@link Types#HEAT}
- {@link Types#COLD}
- other type defined in {@link Types}
- custom - vendor specific type

See Also `LevelData`

142.4.6.1 **public static final String FLOW_IN = "in"**

Represents the metering consumption flow. It can be used as {@link #SERVICE_FLOW} property value.

142.4.6.2 **public static final String FLOW_OUT = "out"**

Represents the metering production flow. It can be used as {@link #SERVICE_FLOW} property value.

142.4.6.3 **public static final String PROPERTY_CURRENT = "current"**

Specifies the current consumption or production property name. The eventable property can be read with {@link #getCurrent()} getter.

142.4.6.4 **public static final String PROPERTY_TOTAL = "total"**

Specifies the total consumption or production property name. The eventable property can be read with {@link #getTotal()} getter.

142.4.6.5 **public static final String SERVICE_FLOW = "dal.meter.flow"**

The service property value contains the metering flow. It's an optional property and available only if it's supported by the meter. The value type is {@code java.lang.String}. Possible property values:

- {@link #FLOW_IN}
- {@link #FLOW_OUT}

142.4.6.6 **public abstract LevelData getCurrent() throws DeviceException**

- Returns the current metering info. It's a getter method for {@link #PROPERTY_CURRENT} property.

Returns The current metering info.

Throws `IllegalStateException` – If this function service object has already been unregistered.

`DeviceException` – If an operation error is available.

See Also `LevelData`

142.4.6.7 **public abstract LevelData getTotal() throws DeviceException**

- Returns the total metering info. It's a getter method for {@link #PROPERTY_TOTAL} property.

Returns The total metering info.

Throws `IllegalStateException` – If this function service object has already been unregistered.

`DeviceException` – If an operation error is available.

See Also `LevelData`

142.4.7 **public abstract MultiLevelControl extends Function**

{@code MultiLevelControl} function provides multi-level control support. The eventable function level is accessible with {@link #getData()} getter and {@link #setData(BigDecimal, String)} setter.

The control type can be:

- {@link Types#LIGHT}
- {@link Types#TEMPERATURE}
- {@link Types#FLOW}
- {@link Types#PRESSURE}
- {@link Types#HUMIDITY}
- {@link Types#GAS}
- {@link Types#SMOKE}
- {@link Types#DOOR}
- {@link Types#WINDOW}
- {@link Types#LIQUID}
- {@link Types#POWER}
- {@link Types#NOISINESS}
- other type defined in {@link Types}
- custom - vendor specific type

See Also `LevelData`

142.4.7.1 **public static final String PROPERTY_DATA = "data"**

Specifies the level property name. The eventable property can be read with {@link #getData()} getter and can be set with {@link #setData(BigDecimal, String)} setters.

142.4.7.2 **public abstract LevelData getData() throws DeviceException**

- Returns {@code MultiLevelControl} level. It's a getter method for {@link #PROPERTY_DATA} property.

Returns `{@code MultiLevelControl}` level.

Throws `IllegalStateException`– If this function service object has already been unregistered.

`DeviceException`– If an operation error is available.

See Also `LevelData`

142.4.7.3 **public abstract void setData(BigDecimal level, String unit) throws DeviceException**

level The new control level.

unit The level unit.

- Sets `{@code MultiLevelControl}` level according to the specified unit. It's a setter method for {@link #PROPERTY_DATA} property.

Throws `IllegalStateException`– If this function service object has already been unregistered.

`DeviceException`– If an operation error is available.

`IllegalArgumentException`– If there is an invalid argument.

142.4.8 **public abstract MultiLevelSensor extends Function**

`{@code MultiLevelSensor}` function provides multi-level sensor monitoring. It reports its state when an important event is available. The eventable state is accessible with {@link #getData()} getter. There are no operations.

The sensor type can be:

- {@link Types#LIGHT}

- {@link Types#TEMPERATURE}
- {@link Types#FLOW}
- {@link Types#PRESSURE}
- {@link Types#HUMIDITY}
- {@link Types#GAS}
- {@link Types#SMOKE}
- {@link Types#DOOR}
- {@link Types#WINDOW}
- {@link Types#LIQUID}
- {@link Types#POWER}
- {@link Types#NOISINESS}
- {@link Types#RAIN}
- other type defined in {@link Types}
- custom - vendor specific type

See Also `LevelData`

142.4.8.1 **public static final String PROPERTY_DATA = "data"**

Specifies the state property name. The eventable property can be read with {@link #getData()} getter.

See Also `LevelData`

142.4.8.2 **public abstract LevelData getData() throws DeviceException**

- Returns the {@code MultiLevelSensor} current state. It's a getter method for {@link #PROPERTY_DATA} property.

Returns The {@code MultiLevelSensor} current state.

Throws `IllegalStateException` – If this function service object has already been unregistered.

`DeviceException` – If an operation error is available.

See Also `LevelData`

142.4.9 **public abstract Types**

Shares common constants for all functions defined in this package. The defined function types are mapped as follow:

- {@link #LIGHT} - {@link MultiLevelControl}, {@link MultiLevelSensor}, {@link BooleanSensor} and {@link BooleanControl}
- {@link #TEMPERATURE} - {@link MultiLevelControl} and {@link MultiLevelSensor}
- {@link #FLOW} - {@link MultiLevelControl} and {@link MultiLevelSensor}
- {@link #PRESSURE} - {@link MultiLevelControl}, {@link MultiLevelSensor} and {@link Meter}
- {@link #HUMIDITY} - {@link MultiLevelControl} and {@link MultiLevelSensor}
- {@link #GAS} - {@link MultiLevelControl}, {@link MultiLevelSensor}, {@link BooleanSensor} and {@link Meter}
- {@link #SMOKE} - {@link MultiLevelControl}, {@link MultiLevelSensor} and {@link BooleanSensor}
- {@link #DOOR} - {@link MultiLevelControl}, {@link MultiLevelSensor}, {@link BooleanSensor} and {@link BooleanControl}
- {@link #WINDOW} - {@link MultiLevelControl}, {@link MultiLevelSensor}, {@link BooleanSensor} and {@link BooleanControl}

- {@link #LIQUID} - {@link MultiLevelControl} and {@link MultiLevelSensor}
- {@link #POWER} - {@link MultiLevelControl}, {@link MultiLevelSensor}, {@link BooleanSensor}, {@link BooleanControl} and {@link Meter}
- {@link #NOISINESS} - {@link MultiLevelControl} and {@link MultiLevelSensor}
- {@link #RAIN} - {@link MultiLevelSensor} and {@link BooleanSensor}
- {@link #CONTACT} - {@link BooleanSensor}
- {@link #FIRE} - {@link BooleanSensor}
- {@link #OCCUPANCY} - {@link BooleanSensor}
- {@link #WATER} - {@link BooleanSensor} and {@link Meter}
- {@link #MOTION} - {@link BooleanSensor}
- {@link #HEAT} - {@link Meter}
- {@link #COLD} - {@link Meter}

The mapping is not mandatory. The function can use custom defined types.

142.4.9.1 **public static final String COLD = "cold"**

The function type is applicable to:

- {@code Meter} - indicates that the {@code Meter} measures thermal energy provided by a source.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.2 **public static final String CONTACT = "contact"**

The function type is applicable to:

- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can detect contact. {@code true} state means that there is contact. {@code false} state means that there is no contact.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.3 **public static final String DOOR = "door"**

The function type is applicable to:

- {@code MultiLevelControl} - indicates that the {@code MultiLevelControl} can control the door position. {@code MultiLevelControl} minimum value can completely close the door. {@code MultiLevelControl} maximum value can open the door to the maximum allowed position.
- {@code MultiLevelSensor} - indicates that the sensor can monitor the door position.
- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can detect the door state. {@code true} state means that the door is opened. {@code false} state means that the door is closed.
- {@code BooleanControl} - indicates that there is a door position control. {@code true} state means that the door will be opened. {@code false} state means that the door will be closed.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.4 **public static final String FIRE = "fire"**

The function type is applicable to:

- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can detect fire. {@code true} state means that there is fire. {@code false} state means that there is no fire.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.5 public static final String FLOW = "flow"

The function type is applicable to:

- `{@code MultiLevelControl}` - indicates that the `{@code MultiLevelControl}` can control the flow level. `{@code MultiLevelControl}` minimum value is the minimum supported flow level. `{@code MultiLevelControl}` maximum value is the maximum supported flow level.
- `{@code MultiLevelSensor}` - indicates that the sensor can monitor the flow level.

This type can be specified as a value of `{@link org.osgi.service.dal.Function#SERVICE_TYPE}`.

142.4.9.6 public static final String GAS = "gas"

The function type is applicable to:

- `{@code MultiLevelControl}` - indicates that the `{@code MultiLevelControl}` can control the gas level. `{@code MultiLevelControl}` minimum value is the lowest supported gas level. `{@code MultiLevelControl}` maximum value is the highest supported gas level.
- `{@code MultiLevelSensor}` - indicates that the sensor can monitor the gas level.
- `{@code BooleanSensor}` - indicates that the `{@code BooleanSensor}` supports gas detection. `{@code true}` state means there is gas. `{@code false}` state means that there is no gas.
- `{@code Meter}` - indicates that the `{@code Meter}` measures the gas consumption.

This type can be specified as a value of `{@link org.osgi.service.dal.Function#SERVICE_TYPE}`.

142.4.9.7 public static final String HEAT = "heat"

The function type is applicable to:

- `{@code Meter}` - indicates that the `{@code Meter}` measures thermal energy provided by a source.

This type can be specified as a value of `{@link org.osgi.service.dal.Function#SERVICE_TYPE}`.

142.4.9.8 public static final String HUMIDITY = "humidity"

The function type is applicable to:

- `{@code MultiLevelControl}` - indicates that the `{@code MultiLevelControl}` can control the humidity level. It's typical functionality for HVAC (heating, ventilation, and air conditioning) devices. `{@code MultiLevelControl}` minimum value is the lowest supported humidity level. `{@code MultiLevelControl}` maximum value is the highest supported humidity level.
- `{@code MultiLevelSensor}` - indicates that the sensor can monitor the humidity level.

This type can be specified as a value of `{@link org.osgi.service.dal.Function#SERVICE_TYPE}`.

142.4.9.9 public static final String LIGHT = "light"

The function type is applicable to:

- `{@code MultiLevelControl}` - indicates that the `{@code MultiLevelControl}` can control light devices. Usually, such devices are called dimmable. `{@code MultiLevelControl}` minimum value can switch off the device and `{@code MultiLevelControl}` maximum value can increase the device light to the maximum possible value.
- `{@code MultiLevelSensor}` - indicates that the sensor can monitor the light level.
- `{@code BooleanSensor}` - indicates that the `{@code BooleanSensor}` can detected light. `{@code true}` state means that there is light. `{@code false}` state means that there is no light.
- `{@code BooleanControl}` - indicates that there is a light device control. `{@code true}` state means that the light device will be turned on. `{@code false}` state means that the light device will be turned off.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.10 **public static final String LIQUID = "liquid"**

The function type is applicable to:

- {@code MultiLevelControl} - indicates that the {@code MultiLevelControl} can control the liquid level. {@code MultiLevelControl} minimum value is the lowest supported liquid level. {@code MultiLevelControl} maximum value is the highest supported liquid level.
- {@code MultiLevelSensor} - indicates that the sensor can monitor the liquid level.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.11 **public static final String MOTION = "motion"**

The function type is applicable to:

- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can detect motion. {@code true} state means that there is motion detection. {@code false} state means that there is no motion detection.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.12 **public static final String NOISINESS = "noisiness"**

The function type is applicable to:

- {@code MultiLevelControl} - indicates that the {@code MultiLevelControl} can control the noise level. {@code MultiLevelControl} minimum value is the lowest supported noise level. {@code MultiLevelControl} maximum value is the highest supported noise level.
- {@code MultiLevelSensor} - indicates that the sensor can monitor the noise level.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.13 **public static final String OCCUPANCY = "occupancy"**

The function type is applicable to:

- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can detect presence. {@code true} state means that someone is detected. {@code false} state means that nobody is detected.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.14 **public static final String POWER = "power"**

The function type is applicable to:

- {@code MultiLevelControl} - indicates that the {@code MultiLevelControl} can control the power level. {@code MultiLevelControl} minimum value is the lowest supported power level. {@code MultiLevelControl} maximum value is the highest supported power level.
- {@code MultiLevelSensor} - indicates that the sensor can monitor the power level.
- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can detect power/no power. {@code true} state means that there is power. {@code false} state means that there is no power.
- {@code BooleanControl} - indicates that there is electricity control. {@code true} state means that the power will be restored. {@code false} state means that the power will be cut.
- {@code Meter} - indicates that the {@code Meter} measures the power consumption.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.15 **public static final String PRESSURE = "pressure"**

The function type is applicable to:

- {@code MultiLevelControl} - indicates that the {@code MultiLevelControl} can control the pressure level. {@code MultiLevelControl} minimum value is the lowest supported pressure level. {@code MultiLevelControl} maximum value is the highest supported pressure level.
- {@code MultiLevelSensor} - indicates that the sensor can monitor the pressure level.
- {@code Meter} - Indicates that the {@code Meter} measures pressure.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.16 **public static final String RAIN = "rain"**

The function type is applicable to:

- {@code MultiLevelSensor} - indicates that the {@code MultiLevelSensor} can monitor the rain rate. It's not applicable to {@code MultiLevelControl}.
- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can detect rain. {@code true} state means that there is rain. {@code false} state means that there is no rain.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.17 **public static final String SMOKE = "smoke"**

The function type is applicable to:

- {@code MultiLevelControl} - indicates that the {@code MultiLevelControl} can control the smoke level. {@code MultiLevelControl} minimum value is the lowest supported smoke level. {@code MultiLevelControl} maximum value is the highest supported smoke level.
- {@code MultiLevelSensor} - indicates that the sensor can monitor the smoke level.
- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can detect smoke. {@code true} state means that there is smoke. {@code false} state means that there is no smoke.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.18 **public static final String TEMPERATURE = "temperature"**

The function type is applicable to:

- {@code MultiLevelControl} - indicates that the {@code MultiLevelControl} can control temperature devices. For example, such device can be thermostat. {@code MultiLevelControl} minimum value is the lowest supported temperature. {@code MultiLevelControl} maximum value is the highest supported temperature.
- {@code MultiLevelSensor} - indicates that the sensor can monitor the temperature.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.19 **public static final String WATER = "water"**

The function type is applicable to:

- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can detect water leak. {@code true} state means that there is water leak. {@code false} state means that there is no water leak.
- {@code Meter} - indicates that the {@code Meter} measures water consumption.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.9.20 **public static final String WINDOW = "window"**

The function type is applicable to:

- {@code MultiLevelControl} - indicates that the {@code MultiLevelControl} can control the window position. {@code MultiLevelControl} minimum value can completely close the window.

{@code MultiLevelControl} maximum value can open the window to the maximum allowed position.

- {@code MultiLevelSensor} - indicates that the sensor can monitor the window position.
- {@code BooleanSensor} - indicates that the {@code BooleanSensor} can window state. {@code true} state means that the window is opened. {@code false} state means that the window is closed.
- {@code BooleanControl} - indicates that there is a window position control. {@code true} state means that the window will be opened. {@code false} state means that the window will be closed.

This type can be specified as a value of {@link org.osgi.service.dal.Function#SERVICE_TYPE}.

142.4.10 **public abstract WakeUp extends Function**

{@code WakeUp} function provides device awake monitoring. It's especially applicable to battery-operated devices. Such device can notify the system that it's awake and can receive commands with a {@link #PROPERTY_AWAKE} property event.

The device can periodically wake up for commands. The interval can be managed with {@link #PROPERTY_WAKE_UP_INTERVAL} property.

See Also `LevelData`, `BooleanData`

142.4.10.1 **public static final String PROPERTY_AWAKE = "awake"**

Specifies the awake property name. The property access type can be {@link PropertyMetadata#ACCESS_EVENTABLE}. If the device is awake, it will trigger a property event.

The property value type is {@code BooleanData}. The boolean data is always {@code true}. It marks that the device is awake.

142.4.10.2 **public static final String PROPERTY_WAKE_UP_INTERVAL = "wakeUpInterval"**

Specifies the wake up interval. The device can periodically wake up and receive commands. That interval is managed by this eventable property. The current property value is available with {@link #getWakeUpInterval()} and can be modified with {@link #setWakeUpInterval(BigDecimal, String)}.

142.4.10.3 **public abstract LevelData getWakeUpInterval() throws DeviceException**

- Returns the current wake up interval. It's a getter method for {@link #PROPERTY_WAKE_UP_INTERVAL} property. The device can periodically wake up and receive command based on this interval.

The interval can be measured in different units like hours, minutes, seconds, etc. The unit is specified in {@code LevelData} instance.

Returns The current wake up interval.

Throws `IllegalStateException` – If this function service object has already been unregistered.

`DeviceException` – If an operation error is available.

See Also `LevelData`

142.4.10.4 **public abstract void setWakeUpInterval(BigDecimal interval, String unit) throws DeviceException**

interval The new wake up interval.

unit The interval unit. If the unit is {@code null}, the interval is measured in milliseconds.

- Sets wake up interval according to the specified unit. It's a setter method for {@link #PROPERTY_WAKE_UP_INTERVAL} property. The device can periodically wake up and receive command based on this interval. The unit can be {@code null}, then the interval is measured in milliseconds.

Throws `IllegalStateException`— If this function service object has already been unregistered.
`DeviceException`— If an operation error is available.
`IllegalArgumentException`— If there is an invalid argument.

142.5 org.osgi.service.dal.functions.data

Device Abstraction Layer Functions Data Package 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.dal.functions.data; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.dal.functions.data; version="[1.0,1.1)"}
```

142.5.1 Summary

- `AlarmData` - Function alarm data.
- `BooleanData` - Function boolean data wrapper.
- `KeypadData` - Represents a keypad event data that is collected when a change with some key from the keypad has occurred.
- `LevelData` - Function level data wrapper.

142.5.2 **public class AlarmData** **extends FunctionData**

Function alarm data. It cares about the alarm type, severity, timestamp and additional metadata. It doesn't support unit. The alarm type is mapped to {@code FunctionData} value.

See Also `Alarm`, `FunctionData`

142.5.2.1 **public static final String FIELD_SEVERITY = "severity"**

Represents the severity field name. The field value is available with {@link #getSeverity()}. The field type is {@code int}. The constant can be used as a key to {@link #AlarmData(Map)}.

142.5.2.2 **public static final String FIELD_TYPE = "type"**

Represents the type field name. The field value is available with {@link #getType()}. The field type is {@code int}. The constant can be used as a key to {@link #AlarmData(Map)}.

142.5.2.3 **public static final int SEVERITY_CRITICAL = 3**

The severity rating indicates that there a critical alarm. The severity priority is higher than {@link #SEVERITY_MINOR} and {@link #SEVERITY_MAJOR}.

142.5.2.4 **public static final int SEVERITY_MAJOR = 2**

The severity rating indicates that there is a major alarm. The severity priority is higher than {@link #SEVERITY_MINOR} and lower than {@link #SEVERITY_CRITICAL}.

142.5.2.5 **public static final int SEVERITY_MINOR = 1**

The severity rating indicates that there is a minor alarm. The severity priority is lower than {@link #SEVERITY_MAJOR} and {@link #SEVERITY_CRITICAL}.

142.5.2.6 public static final int SEVERITY_UNDEFINED = 0

The severity constant indicates that there is no severity rating for this alarm.

142.5.2.7 public static final int TYPE_ACCESS_CONTROL = 1

The alarm type indicates that there is access control issue. For example, the alarm can indicate that the door is unlocked.

142.5.2.8 public static final int TYPE_BURGLAR = 2

The alarm type indicates that there is a burglar notification. For example, the alarm can indicate that the glass is broken.

142.5.2.9 public static final int TYPE_COLD = 3

The alarm type indicates that temperature is too low.

142.5.2.10 public static final int TYPE_GAS_CO = 4

The alarm type indicates that carbon monoxide (CO) is detected.

142.5.2.11 public static final int TYPE_GAS_CO2 = 5

The alarm type indicates that carbon dioxide (CO2) is detected.

142.5.2.12 public static final int TYPE_HARDWARE_FAIL = 7

The alarm type indicates that there is hardware failure.

142.5.2.13 public static final int TYPE_HEAT = 6

The alarm type indicates that temperature is too high.

142.5.2.14 public static final int TYPE_POWER_FAIL = 8

The alarm type indicates a power cut.

142.5.2.15 public static final int TYPE_SMOKE = 9

The alarm type indicates that smoke is detected.

142.5.2.16 public static final int TYPE_SOFTWARE_FAIL = 10

The alarm type indicates that there is software failure.

142.5.2.17 public static final int TYPE_TAMPER = 11

The alarm type for a tamper indication.

142.5.2.18 public static final int TYPE_UNDEFINED = 0

The alarm type indicates that the type is not specified.

142.5.2.19 public static final int TYPE_WATER = 12

The alarm type indicates that a water leak is detected.

142.5.2.20 public AlarmData(Map<String, ?> fields)

fields Contains the new {@code AlarmData} instance field values.

- Constructs new {@code AlarmData} instance with the specified field values. The map keys must match to the field names. The map values will be assigned to the appropriate class fields. For example, the maps can be: {"severity"=Integer(1)...}. That map will initialize the {@link #FIELD_SEVERITY} field with 1. If severity is missing, {@link #SEVERITY_UNDEFINED} is used.
- {@link #FIELD_SEVERITY} - optional field. The value type must be {@code Integer}.

- {@link #FIELD_TYPE} - optional field. The value type must be {@code Integer}.

Throws `ClassCastException`– If the field value types are not expected.

`IllegalArgumentException`– If the alarm severity is invalid.

`NullPointerException`– If the fields map is {@code null}.

142.5.2.21 **public AlarmData(long timestamp, Map<String, ?> metadata, int severity, int type)**

timestamp The alarm data timestamp optional field.

metadata The alarm data metadata optional field.

severity The alarm data severity optional field.

type The alarm data type optional field.

- Constructs new {@code AlarmData} instance with the specified arguments.

Throws `IllegalArgumentException`– If the alarm severity is invalid.

142.5.2.22 **public int compareTo(Object o)**

o {@code AlarmData} to be compared.

- Compares this {@code AlarmData} instance with the given argument. If the argument is not {@code AlarmData}, it throws {@code ClassCastException}. Otherwise, this method returns:

- {@code -1} if this instance field is less than a field of the specified argument.
- {@code 0} if all fields are equivalent.
- {@code 1} if this instance field is greater than a field of the specified argument.

The fields are compared in this order: timestamp, metadata, type, severity.

Returns {@code -1}, {@code 0} or {@code 1} depending on the comparison rules.

Throws `ClassCastException`– If the method argument is not of type {@code AlarmData}.

See Also `java.lang.Comparable.compareTo(java.lang.Object)`

142.5.2.23 **public boolean equals(Object o)**

o The object to compare this data.

- Two {@code AlarmData} instances are equal if they contain equal metadata, timestamp, type and severity.

Returns {@code true} if this object is equivalent to the specified one.

See Also `org.osgi.service.dal.FunctionData.equals(java.lang.Object)`

142.5.2.24 **public int getSeverity()**

- Returns the alarm severity. The severity can be one of:

- {@link #SEVERITY_UNDEFINED}
- {@link #SEVERITY_MINOR}
- {@link #SEVERITY_MAJOR}
- {@link #SEVERITY_CRITICAL}

Returns The alarm severity.

142.5.2.25 **public int getType()**

- Returns the alarm type. The type can be one of the predefined:

- {@link #TYPE_UNDEFINED}

- {@link #TYPE_SMOKE}
- {@link #TYPE_HEAT}
- {@link #TYPE_COLD}
- {@link #TYPE_GAS_CO}
- {@link #TYPE_GAS_CO2}
- {@link #TYPE_WATER}
- {@link #TYPE_POWER_FAIL}
- {@link #TYPE_HARDWARE_FAIL}
- {@link #TYPE_SOFTWARE_FAIL}
- vendor specific

Zero and positive values are reserved for this definition and further extensions of the alarm types. Custom types can be used only as negative values to prevent potential collisions.

Returns The alarm type.

142.5.2.26 **public int hashCode()**

- Returns the hash code for this {@code AlarmData} object. The hash code is a sum of {@link FunctionData#hashCode()}, the alarm severity and the alarm type.

Returns The hash code of this {@code AlarmData} object.

See Also org.osgi.service.dal.FunctionData.hashCode()

142.5.2.27 **public String toString()**

- Returns the string representation of this alarm data.

Returns The string representation of this alarm data.

142.5.3 **public class BooleanData extends FunctionData**

Function boolean data wrapper. It can contain a boolean value, timestamp and additional metadata. It doesn't support measurement unit.

See Also BooleanControl, BooleanSensor, FunctionData

142.5.3.1 **public static final String FIELD_VALUE = "value"**

Represents the value field name. The field value is available with {@link #getValue()}. The field type is {@code boolean}. The constant can be used as a key to {@link #BooleanData(Map)}.

142.5.3.2 **public BooleanData(Map<String, ?> fields)**

fields Contains the new {@code BooleanData} instance field values.

- Constructs new {@code BooleanData} instance with the specified field values. The map keys must match to the field names. The map values will be assigned to the appropriate class fields. For example, the maps can be: {"value"=Boolean(true)...}. That map will initialize the {@link #FIELD_VALUE} field with {@code true}.

{@link #FIELD_VALUE} - mandatory field. The value type must be {@code Boolean}.

Throws ClassCastException – If the field value types are not expected.

IllegalArgumentException – If the value is missing.

NullPointerException – If the fields map is {@code null}.

142.5.3.3 **public BooleanData(long timestamp, Map<String, ?> metadata, boolean value)**

timestamp The boolean data timestamp optional field.

metadata The boolean data metadata optional field.

value The boolean value mandatory field.

- Constructs new {@code BooleanData} instance with the specified arguments.

142.5.3.4 **public int compareTo(Object o)**

o {@code BooleanData} to be compared.

- Compares this {@code BooleanData} instance with the given argument. If the argument is not {@code BooleanData}, it throws {@code ClassCastException}. Otherwise, this method returns:
 - {@code -1} if this instance field is less than a field of the specified argument.
 - {@code 0} if all fields are equivalent.
 - {@code 1} if this instance field is greater than a field of the specified argument.

The fields are compared in this order: timestamp, metadata, value.

Returns {@code -1}, {@code 0} or {@code 1} depending on the comparison rules.

Throws `ClassCastException`— If the method argument is not of type {@code BooleanData}.

See Also `java.lang.Comparable.compareTo(java.lang.Object)`

142.5.3.5 **public boolean equals(Object o)**

o The object to compare this data.

- Two {@code BooleanData} instances are equal if they contain equal metadata, timestamp and boolean value.

Returns {@code true} if this object is equivalent to the specified one.

See Also `org.osgi.service.dal.FunctionData.equals(java.lang.Object)`

142.5.3.6 **public boolean getValue()**

- Returns {@code BooleanData} value.

Returns {@code BooleanData} value.

142.5.3.7 **public int hashCode()**

- Returns the hash code for this {@code BooleanData} object. The hash code is a sum of {@code FunctionData.hashCode()} and {@code Boolean.hashCode()}, where {@code Boolean.hashCode()} represents the boolean value hash code.

Returns The hash code of this {@code BooleanData} object.

See Also `org.osgi.service.dal.FunctionData.hashCode()`

142.5.3.8 **public String toString()**

- Returns the string representation of this boolean data.

Returns The string representation of this boolean data.

142.5.4 **public class KeypadData extends FunctionData**

Represents a keypad event data that is collected when a change with some key from the keypad has occurred.

The key pressed event is using {@code #TYPE_PRESSED} type, while the key released event is using {@code #TYPE_RELEASED} type.

See Also Keypad, FunctionData

142.5.4.1 public static final String FIELD_KEY_CODE = "keyCode"

Represents the key code field name. The field value is available with {@link #getKeyCode()}. The field type is {@code int}. The constant can be used as a key to {@link #KeypadData(Map)}.

142.5.4.2 public static final String FIELD_KEY_NAME = "keyName"

Represents the key name field name. The field value is available with {@link #getKeyName()}. The field type is {@code String}. The constant can be used as a key to {@link #KeypadData(Map)}.

142.5.4.3 public static final String FIELD_SUB_TYPE = "subType"

Represents the event sub-type field name. The field value is available with {@link #getSubType()}. The field type is {@code int}. The constant can be used as a key to {@link #KeypadData(Map)}.

142.5.4.4 public static final String FIELD_TYPE = "type"

Represents the event type field name. The field value is available with {@link #getType()}. The field type is {@code int}. The constant can be used as a key to {@link #KeypadData(Map)}.

142.5.4.5 public static final int SUB_TYPE_PRESSED_DOUBLE = 3

Represents a keypad event sub-type for a double key pressed event. Usually, there are two press actions and the key is not held down after the second press. This sub-type is used with {@link #TYPE_PRESSED} type.

142.5.4.6 public static final int SUB_TYPE_PRESSED_DOUBLE_LONG = 4

Represents a keypad event sub-type for a double long key pressed event. Usually, there are two press actions and the key is held down after the second press. This sub-type is used with {@link #TYPE_PRESSED} type.

142.5.4.7 public static final int SUB_TYPE_PRESSED_LONG = 2

Represents a keypad event sub-type for a long key pressed event. Usually, there is a single press and the key is held down. This sub-type is used with {@link #TYPE_PRESSED} type.

142.5.4.8 public static final int SUB_TYPE_PRESSED_NORMAL = 1

Represents a keypad event sub-type for a normal key pressed event. Usually, there is a single press and the key is not held down. This sub-type is used with {@link #TYPE_PRESSED} type.

142.5.4.9 public static final int TYPE_PRESSED = 0

Represents a keypad event type for a key pressed event.

142.5.4.10 public static final int TYPE_RELEASED = 1

Represents a keypad event type for a key released event.

142.5.4.11 public KeypadData(Map<String, ?> fields)

fields Contains the new {@code KeypadData} instance field values.

- Constructs new {@code KeypadData} instance with the specified field values. The map keys must match to the field names. The map values will be assigned to the appropriate class fields. For example, the maps can be: {"type"=Integer(1)...}. That map will initialize the {@link #FIELD_TYPE} field with 1.
 - {@link #FIELD_TYPE} - mandatory field. The value type must be {@code Integer}.
 - {@link #FIELD_SUB_TYPE} - optional field. The value type must be {@code Integer}.
 - {@link #FIELD_KEY_CODE} - mandatory field. The value type must be {@code Integer}.

- {@link #FIELD_KEY_NAME} - optional field. The value type must be {@code String}.

Throws `ClassCastException`— If the field value types are not expected.

`IllegalArgumentException`— If the event type or key code is missing or invalid arguments are specified.

`NullPointerException`— If the fields map is {@code null}.

142.5.4.12 **public KeypadData(long timestamp, Map<String, Object> metadata, int type, int subType, int keyCode, String keyName)**

timestamp The data timestamp optional field.

metadata The data metadata optional field.

type The data event type mandatory field.

subType The data event sub-type optional field or {@code 0} if there is no sub-type.

keyCode The data key code mandatory field.

keyName The data key name optional field or {@code null} if there is no key name.

- Constructs new {@code KeypadData} instance with the specified arguments.

142.5.4.13 **public int compareTo(Object o)**

o {@code KeypadData} to be compared.

- Compares this {@code KeypadData} instance with the given argument. If the argument is not {@code KeypadData}, it throws {@code ClassCastException}. Otherwise, this method returns:

- {@code -1} if this instance field is less than a field of the specified argument.
- {@code 0} if all fields are equivalent.
- {@code 1} if this instance field is greater than a field of the specified argument.

The fields are compared in this order: timestamp, metadata, type, sub-type, key code, key name.

Returns {@code -1}, {@code 0} or {@code 1} depending on the comparison rules.

Throws `ClassCastException`— If the method argument is not of type {@code KeypadData}.

See Also `java.lang.Comparable.compareTo(java.lang.Object)`

142.5.4.14 **public boolean equals(Object o)**

o The object to compare this data.

- Two {@code KeypadData} instances are equal if they contain equal metadata, timestamp, event type, key code and key name.

Returns {@code true} if this object is equivalent to the specified one.

See Also `org.osgi.service.dal.FunctionData.equals(java.lang.Object)`

142.5.4.15 **public int getKeyCode()**

- The code of the key. This field is mandatory and it holds the semantics(meaning) of the key.

Returns The key code.

142.5.4.16 **public String getKeyName()**

- Represents a human readable name of the corresponding key code. This field is optional and sometimes it could be missed(might be {@code null}).

Returns A string with the name of the key or {@code null} if not specified.

142.5.4.17 public int getSubType()

- Returns the event sub-type. The sub-type provides additional details about the event. The sub-type can be one of:
 - {@link #SUB_TYPE_PRESSED_NORMAL}
 - {@link #SUB_TYPE_PRESSED_LONG}
 - {@link #SUB_TYPE_PRESSED_DOUBLE}
 - {@link #SUB_TYPE_PRESSED_DOUBLE_LONG}
 - custom sub-type

Zero and positive values are reserved for this definition and further extensions of the sub-types. Custom sub-types can be used only as negative values to prevent potential collisions.

Returns The event sub-type.

142.5.4.18 public int getType()

- Returns the event type. The type represents the main reason for this event. It can be one of:
 - {@link #TYPE_PRESSED}
 - {@link #TYPE_RELEASED}

Returns The event type.

142.5.4.19 public int hashCode()

- Returns the hash code for this {@code KeypadData} object. The hash code is a sum of {@link FunctionData#hashCode()}, {@link String#hashCode()}, event type, event sub-type and key code, where {@link String#hashCode()} represents the key name hash code if available.

Returns The hash code of this {@code LevelData} object.

See Also org.osgi.service.dal.FunctionData.hashCode()

142.5.4.20 public String toString()

- Returns the string representation of this keypad data.

Returns The string representation of this keypad data.

142.5.5 public class LevelData extends FunctionData

Function level data wrapper. It supports all properties defined in {@code FunctionData}.

See Also MultiLevelControl, MultiLevelSensor, Meter, FunctionData

142.5.5.1 public static final String FIELD_LEVEL = "level"

Represents the level field name. The field value is available with {@link #getLevel()}. The field type is {@code BigDecimal}. The constant can be used as a key to {@link #LevelData(Map)}.

142.5.5.2 public static final String FIELD_UNIT = "unit"

Represents the unit field name. The field value is available with {@link #getUnit()}. The field type is {@code String}. The constant can be used as a key to {@link #LevelData(Map)}.

142.5.5.3 public LevelData(Map<String, ?> fields)

fields Contains the new {@code LevelData} instance field values.

- Constructs new {@code LevelData} instance with the specified field values. The map keys must match to the field names. The map values will be assigned to the appropriate class fields. For exam-

ple, the maps can be: {"level"=BigDecimal(1)...}. That map will initialize the {@link #FIELD_LEVEL} field with 1.

- {@link #FIELD_LEVEL} - mandatory field. The value type must be {@code BigDecimal}.
- {@link #FIELD_UNIT} - optional field. The value type must be {@code String}.

Throws `ClassCastException` – If the field value types are not expected.

`IllegalArgumentException` – If the level is missing.

`NullPointerException` – If the fields map is {@code null}.

142.5.5.4 **public LevelData(long timestamp, Map<String, Object> metadata, BigDecimal level, String unit)**

timestamp The data timestamp optional field.

metadata The data metadata optional field.

level The level value mandatory field.

unit The data unit optional field.

- Constructs new {@code LevelData} instance with the specified arguments.

Throws `NullPointerException` – If {@code level} is {@code null}.

142.5.5.5 **public int compareTo(Object o)**

o {@code LevelData} to be compared.

- Compares this {@code LevelData} instance with the given argument. If the argument is not {@code LevelData}, it throws {@code ClassCastException}. Otherwise, this method returns:
 - {@code -1} if this instance field is less than a field of the specified argument.
 - {@code 0} if all fields are equivalent.
 - {@code 1} if this instance field is greater than a field of the specified argument.

The fields are compared in this order: timestamp, metadata, level, unit.

Returns {@code -1}, {@code 0} or {@code 1} depending on the comparison rules.

Throws `ClassCastException` – If the method argument is not of type {@code LevelData}.

See Also `java.lang.Comparable.compareTo(java.lang.Object)`

142.5.5.6 **public boolean equals(Object o)**

o The object to compare this data.

- Two {@code LevelData} instances are equal if they contain equal metadata, timestamp, unit and level.

Returns {@code true} if this object is equivalent to the specified one.

See Also `org.osgi.service.dal.FunctionData.equals(java.lang.Object)`

142.5.5.7 **public BigDecimal getLevel()**

- Returns {@code LevelData} value. The value type is {@code BigDecimal} instead of {@code double} to guarantee value accuracy.

Returns The {@code LevelData} value.

142.5.5.8 **public String getUnit()**

- Returns {@code LevelData} unit as it's specified in {@link PropertyMetadata#UNITS} or {@code null} if the unit is missing.

Returns The value unit or {@code null} if the unit is missing.

142.5.5.9 public int hashCode()

- Returns the hash code for this {@code LevelData} object. The hash code is a sum of {@code {@link FunctionData#hashCode()}}, {@code {@link String#hashCode()}} and {@code {@link BigDecimal#hashCode()}}, where {@code {@link String#hashCode()}} represents the unit hash code and {@code {@link BigDecimal#hashCode()}} represents the level hash code.

Returns The hash code of this {@code LevelData} object.

See Also org.osgi.service.dal.FunctionData.hashCode()

142.5.5.10 public String toString()

- Returns the string representation of this level data.

Returns The string representation of this level data.

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT